

SIMPLE STUDENT RESULT MANAGEMENT SYSTEM

End-of-Semester C# Project Report

Group Eight

Group Members

No.	Name
1	Akena Ismail
2	Ssekanjako Jamal
3	Muwanguzi Travis
4	Tata Noel James
5	Emaset Terry
6	Arocha Gedion
7	Amazu Norman

1. Introduction

This report is about the Simple Student Result Management System that I developed as my End-of-Semester project for C# Programming. I built it as a C# Console Application, following the requirements given in our project brief. The main aim of this project was to show that I can use the things we learnt in class this semester, like variables, input and output, if statements, switch statements, loops, methods, and lists.

The program is meant to help a lecturer enter a student's details and marks, and the program does the calculations on its own instead of the lecturer calculating by hand. It gives the final mark, the grade, and tells the lecturer if the student passed or failed. It can also show statistics for a whole class of students.

2. Problem Statement

In most training institutions, lecturers record and calculate student results manually, or using Excel. This can take a lot of time and it is easy to make a mistake, especially when calculating grades for many students or adding up class averages. There is a need for a simple program that can:

- Record a student's details and marks correctly.
- Calculate the final mark and grade automatically so there are no calculation errors.
- Stop the lecturer from entering wrong marks, for example a coursework mark above 40.
- Show statistics for the whole class quickly, like the average mark and how many students passed.

This is the problem that my project is trying to solve, by giving a simple console program that can do all of this for a lecturer.

3. Objectives

The objectives of my project, based on the assignment brief, were to show that I can:

- Use variables and data types like string, int, double and bool.
- Get input from the user and display output on the screen.
- Use arithmetic, comparison and logical operators.
- Use if, if...else, else if and switch statements to make decisions in the program.
- Use loops such as for, while, do...while and foreach.
- Use a List<T> to store more than one student's marks.
- Create and use methods so that my code is organised and not all in one place.
- Validate user input so that the program does not crash or accept wrong marks.
- Apply problem-solving skills to design a program that actually works.

4. System Requirements

4.1 Functional Requirements

These are the things the program is able to do:

- Allow the lecturer to enter the student's registration number, name, gender, programme, and course.
- Allow the lecturer to enter coursework marks (0–40) and examination marks (0–60).
- Calculate the Final Mark as Coursework + Examination.
- Work out the grade (A–F) and the remark based on the final mark.
- Decide if the student has passed or failed (Pass: 40 and above, Fail: below 40).
- Show one student's result in a neat report format.
- Calculate and show class statistics: number of students, highest mark, lowest mark, average mark, number who passed and failed, and the pass/fail percentages.
- Check that marks entered are within the correct range and reject them if not.
- Not crash if the lecturer types letters instead of numbers by mistake.

4.2 Non-Functional Requirements

- The program runs as a C# Console Application.
- The program is easy to use through a simple numbered menu.

- I used meaningful variable and method names and added comments in the code where I felt it was necessary.

4.3 Grading Scale

Final Mark	Grade	Remark
80–100	A	Excellent
70–79	B	Very Good
60–69	C	Good
50–59	D	Pass
40–49	E	Weak Pass
0–39	F	Fail

5. Program Design

I designed the program around a main menu that keeps showing until the lecturer chooses to exit. Each time the lecturer picks an option, the program calls a method that does that specific job. I store the current student's information in variables, and once I finish calculating a student's result, I save it into Lists so that I can use it later for the class statistics.

5.1 Main Menu

```
=====
STUDENT RESULT MANAGEMENT SYSTEM
=====
1. Enter Student Details
2. Enter Student Marks
3. Calculate Final Mark
4. Display Student Result
5. Display Class Statistics
6. Exit
Enter your choice:
```

5.2 Data Structures Used

- string — for text like the name, registration number, gender, programme, and course.
- int — for whole number marks (coursework, examination, final mark).
- double — for the average and the percentages, since these are not whole numbers.
- bool — to keep track of things like whether the student's details have already been entered.
- List<T> — I used this to store the marks and statuses of every student I enter, because I did not know in advance how many students the lecturer would enter.

5.3 Program Flow (Design Logic)

1. Show the menu in a loop until the user chooses Exit.
2. Enter Student Details – save the bio-data of the current student.
3. Enter Marks – check and save the coursework and examination marks.
4. Calculate Final Mark – add the marks together, work out the grade and remark using a switch statement, decide pass or fail using a comparison operator, then save the finished record into the class Lists.
5. Display Result – print the report for the current student.
6. Display Statistics – go through the class Lists to work out the highest, lowest, average, and pass/fail numbers.

6. Explanation of the C# Code

6.1 Input Validation

To validate the coursework and examination marks, I used a loop together with `int.TryParse()`. I chose `TryParse` instead of `Convert.ToInt32()` because `TryParse` does not crash the program if the user types letters instead of numbers — it just returns false and I can ask again. If the number entered is outside the allowed range, the loop asks the lecturer to enter it again:

```
while (true)
{
```

```

    if (!int.TryParse(input, out coursework))
    { Console.WriteLine("Invalid input!"); continue; }
    if (coursework < 0 || coursework > 40)
    { Console.WriteLine("Must be 0-40."); continue; }
    break;
}

```

6.2 Grade Calculation (switch statement)

In my CalculateGrade() method, I divide the mark by 10 first. This groups the marks into tens, for example a mark of 77 becomes 7, and a mark of 89 becomes 8. Then I use a switch statement to check that number and return the correct grade:

```

switch (mark / 10)
{
    case 10: case 9: case 8: return "A";
    case 7: return "B";
    case 6: return "C";
    case 5: return "D";
    case 4: return "E";
    default: return "F";
}

```

6.3 Pass/Fail Decision (comparison operator)

To decide if a student passed or failed, I just compare the final mark to 40 using the comparison operator >=, like this: status = (finalMark >= 40) ? "PASS" : "FAIL";

6.4 Class Statistics (loops and Lists)

In my DisplayStatistics() method, I use a foreach loop to go through the List<int> that holds all the final marks I have saved. As it loops, it adds to a running total and checks if the current mark is higher or lower than what is already stored as the highest or lowest. After the loop, I divide the total by the number of students to get the average:

```

foreach (int m in marks)
{
    total += m;
    if (m > highest) highest = m;
    if (m < lowest) lowest = m;
}
double average = (double)total / marks.Count;

```

6.5 Methods I Created

Method	What it does
GetValidMark()	Keeps asking for a mark until the lecturer enters a valid one within the range.
CalculateFinalMark()	Adds the coursework and examination marks together to get the final mark.
CalculateGrade()	Works out the letter grade from the final mark using a switch statement.
GetRemark()	Works out the remark (like "Very Good") that matches the grade.
DisplayResult()	Prints the report for one student.
DisplayStatistics()	Goes through all the students I have recorded and prints the class statistics.

7. Testing

I tested my program by running it several times and entering different values, including some wrong values on purpose, to check that it behaves the way it is supposed to. [Fill in this table with your own actual results before you submit.]

Test Case	Input	Expected Result	Actual Result
Valid marks	Coursework: 32, Exam: 45	Final Mark: 77, Grade: B, Status: PASS	[Fill in]
Coursework above range	Coursework: 55	"Invalid mark! Coursework must be between 0 and 40."	[Fill in]
Examination above	Exam: 75	"Invalid mark! Examination must be between 0 and 40."	[Fill in]

range		be between 0 and 60."	
Non-numeric mark entered	Coursework: "abc"	"Invalid input! Please enter a whole number." (no crash)	[Fill in]
Boundary pass mark	Final Mark: 40	Grade: E, Status: PASS	[Fill in]
Boundary fail mark	Final Mark: 39	Grade: F, Status: FAIL	[Fill in]
Multiple students	Enter 3+ students	Class statistics reflect all entered students	[Fill in]

8. Screenshots

[Paste your own screenshots here of: (1) the main menu, (2) entering student details, (3) entering marks, including one screenshot where you enter a wrong mark on purpose to show the validation message, (4) the student result screen, and (5) the class statistics screen.]

9. Challenges Encountered

[Replace this with the challenges you personally faced. Some examples of things students often struggle with on this project:]

- Getting the program to not crash when a letter is typed instead of a number — I solved this by using `int.TryParse()` instead of `Convert.ToInt32()`.
- Deciding whether to use an array or a `List<T>` for storing the students, since I did not know beforehand how many students the lecturer would enter.
- Breaking my code into separate methods so that the Main part of my program does not become too long and confusing.
- Making sure marks right on the boundary, like exactly 40 or exactly 80, are graded correctly.

10. Conclusion

Overall, I managed to build a Student Result Management System that does what was asked in the project brief. Through this project I was able to practice using variables, operators, if and switch statements, loops, methods, and `List<T>`, and apply them to a problem that lecturers actually face. My program checks the marks entered so that it does not crash or accept wrong values, and it gives correct results for both a single student and for a whole class. If I get more time, I would like to add features like searching for a student, saving the results to a file, and maybe building it with a graphical interface using Windows Forms instead of the console.